# PyBullet Collision / Overlap 判定設計まとめ

本ドキュメントは、直近の議論を整理し、**PyBullet における collision / overlap 判定を kinematics / physics の両観点でどう設計・実装するか**をまとめたものです。 VSCode + Codex でのリファクタリング指針として使うことを想定しています。

---

## 1. 前提と目的

### 想定ユースケース

- 主用途：**kinematics 主体の衝突回避・安全チェック**
- 副用途：**physics を有効にした接触挙動の検証・再現**
- 対象：
- 複数ロボット / 複数障害物
- URDF / MultiBody / link 構造あり
- 大規模（N 増大）・高速（擬似 Nx speed）シミュレーション

### 重要な設計方針

- 「当たったか？」より **「当たりそうか？」** を重視
- 再現性・安定性・スケール性を優先
- Physics は 必要なときだけ 使う

---

## 2. stepSimulation() の役割

`stepSimulation()` は単なる update 関数ではなく、以下をまとめて実行する：

1. 剛体の時間積分（位置・姿勢・速度）
2. Broadphase（衝突候補探索）
3. Narrowphase（衝突判定）
4. 接触マニフォールド生成
5. 接触・拘束の解決（押し返し・摩擦）
6. 接触情報のキャッシュ更新

### 重要ポイント

- `getContactPoints()` が返す情報は **stepSimulation の結果**
- stepSimulation を呼ばない場合、contact は
- 古い
- 欠落する
- タイミング依存

---

## 3. stepSimulation は毎回必須か？

### 必須ではないケース（Physics OFF）

- 完全 kinematics 運用
- resetBasePositionAndOrientation / resetJointState で姿勢更新
- 押し返し・摩擦・質量効果が不要

この場合： - stepSimulation を呼ばなくても良い - 判定は `getClosestPoints` / `getAABB` が安定

### 必須なケース（Physics ON）

- 動的物体（mass > 0）が存在
- 実際の接触挙動を扱いたい
- `getContactPoints` を信頼したい

この場合： - 固定 Δt で stepSimulation を周期実行する前提が必要

---

## 4. Collision API の性質整理

### getContactPoints

- 意味：**物理エンジンが「接触」と認識した結果**
- 特徴：
- 接触していないと基本的に空
- すれすれ / 安全距離は分からない
- solver・step タイミング依存
- 向いている用途：
- physics–physics の実接触ログ
- デバッグ・事故再現

### getClosestPoints

- 意味：**幾何的な最近接距離クエリ**
- 特徴：
- stepSimulation 非依存
- reset 移動でも安定
- distance > 0 を使えば安全距離評価が可能
- 向いている用途：
- kinematics 主体の衝突回避
- クリアランス / safety margin 判定
- physics / kinematics 混在

---

## 5. Kinematics object と getContactPoints

- kinematics object（mass=0, reset 移動）でも
- 条件が揃えば getContactPoints が返ることはある
- ただし：
- kinematic–kinematic では不安定
- わずかなめり込みは検出されないことがある

結論： - **kinematics 主体の判定に getContactPoints を使うのは危険** - 判定の主軸は getClosestPoints

---

## 6. AABB → Narrow-phase の基本構成

推奨フロー：

1. **Broad-phase（候補削減）**
2. getAABB
3. 自前 spatial hash / grid
4. getOverlappingObjects（1体近傍探索用）
5. **Narrow-phase（確定判定）**
6. kinematics：`getClosestPoints(distance=margin)`
7. physics 接触ログ：`getContactPoints()`

※ 全ペアに直接 closest/contact を投げない

---

## 7. getOverlappingObjects vs 自前 AABB

### getOverlappingObjects が向くケース

- 特定の 1 体（例：フォーク先端）の近傍探索
- 少数クエリ

### 自前 AABB / grid が向くケース

- 全体の衝突ペア列挙
- 多数ボディ（N=100〜1000）
- 毎 step 実行

---

## 8. ベンチマーク結果の解釈指針

- getContactPoints が速く出るのは自然なケースが多い
- stepSimulation 後
- 非接触ペアが多い
- contact manifold が既に構築済み

ただし： - 速度が速い ≠ 要件に合っている - kinematics / 安全距離評価では closest が必要

**性能比較は「同一 world / 同一状態 / 同一候補ペア」で行うこと**

---

## 9. 推奨する二層モード設計

### Physics OFF モード（デフォルト）

- stepSimulation：呼ばない or 低頻度
- 姿勢更新：reset 系 API
- 衝突判定：getClosestPoints(distance=margin)
- 特徴：高速・再現性高・Nx speed 向き

### Physics ON モード（検証用）

- stepSimulation：固定 Δt で周期実行
- 衝突判定：
- 判定：getClosestPoints(distance=margin)
- 実接触ログ：getContactPoints()
- 特徴：実挙動再現・デバッグ向き

---

## 10. 実装パラメータ例

推奨フラグ： - physics_enabled: bool - collision_query_mode: {"closest", "contact", "hybrid"} - collision_margin: float # 0.0 = 接触のみ

推奨デフォルト： - physics_enabled = False - collision_query_mode = "closest" - collision_margin = 0.02〜0.05

---

## 11. 結論

- stepSimulation を呼ばなくても PyBullet を使う価値は **十分にある**
- PyBullet は
- Physics Engine
- Collision Geometry Engine の両面を持つ

設計としては：

**通常運用：Physics OFF + getClosestPoints 検証/再現：Physics ON + getContactPoints**

が最も破綻しにくく、スケールしやすい。

---

（このドキュメントは Codex によるリファクタリング指針として利用可能）